Sorting Algorithms

Quick Sort

Core Idea

Pick an element called the **pivot**, then **partition** the array so that every element smaller than the pivot goes to its left and every element larger goes to its right. The pivot is now in its **exact final position**. Recursively repeat this on the left and right sides.

Steps

1. If the array has 1 or 0 elements — **return** (base case)
2. Pick a **pivot** element
3. **Partition** — move all elements smaller than pivot to its left, larger to its right
4. Pivot is now in its correct final position
5. Recursively apply quick sort on the **left side**
6. Recursively apply quick sort on the **right side**

Visualizing It

`[5, 3, 8, 1]` — pivot = `5` (first element)

Partition:

- 3 < 5 → goes left
- 8 > 5 → goes right
- 1 < 5 → goes left

Result: `[3, 1, 5, 8]` → pivot `5` is in its final position ✅

Recurse left `[3, 1]` → pivot = `3`

- 1 < 3 → goes left
- Result: `[1, 3]` ✅

Recurse right `[8]` → single element, done ✅

Final: `[1, 3, 5, 8]` ✅

Key Characteristics

- **Not Stable** — partitioning can disturb original order of equal elements
- **In-place** — no extra array needed, partitioning happens within the array
- **Recursive** — uses the call stack naturally

Time & Space Complexity

Case	Time	Why
Best	O(n log n)	Pivot always splits array into two equal halves
Average	O(n log n)	Reasonably balanced splits on average
Worst	$O(n^2)$	Pivot is always the smallest or largest element

Space: O(log n) — no extra array, but recursion uses the call stack

---

The Optimization — Pivot Selection

The entire performance of Quick Sort depends on **how you pick the pivot**. Three common strategies:

- **First/Last element** — simplest, but terrible on already sorted arrays → triggers $O(n^2)$
- **Random element** — good in practice, avoids predictable worst cases
- **Median of three** — pick the median of first, middle, and last element — most commonly used in real implementations

---

Quick Sort vs Merge Sort

	Quick Sort	Merge Sort
Worst case	$O(n^2)$	O(n log n)
Space	O(log n)	O(n)
Stable	No	Yes
In-place	Yes	No
Speed in practice	**Faster**	Slower

Quick Sort is faster in practice despite the worse worst case because it has **better cache performance** — it works within the original array rather than creating new ones.

Why It's The Most Asked in Interviews

It combines **recursion, divide and conquer, and partitioning logic** all in one — making it a goldmine for testing problem solving depth. Interviewers love asking about pivot strategy, worst case scenarios, and how it compares to Merge Sort. Always know all three.

The issue is pivot selection — always choosing the first element as pivot.

Why You're Getting TLE

The issue is **pivot selection — always choosing the first element as pivot**.

When the array is already sorted (or nearly sorted), this creates **worst-case $O(n^2)$** behavior:

```
[1, 2, 3, 4, 5]
pivot=1 → left=[], right=[2,3,4,5]   # unbalanced!
pivot=2 → left=[], right=[3,4,5]      # unbalanced again!
pivot=3 → left=[], right=[4,5]        # every single time...
```

Instead of O(n log n) splits, you get O(n) levels → **$O(n^2)$ total**.

> Your "smaller side first" optimization doesn't help here because the imbalance itself is the problem — you can't optimize around a pivot that's always wrong.

Fix: Randomize the Pivot

```
import random

class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        self.quickSort(nums, 0, len(nums) - 1)
        return nums

    def quickSort(self, nums, startIdx, endIdx):
        if startIdx >= endIdx:
            return

        # Randomly pick pivot and swap it to the start
        randIdx = random.randint(startIdx, endIdx)
        self.swap(nums, startIdx, randIdx)

        pivot = startIdx
        left = startIdx + 1
        right = endIdx

        while left <= right:
            if nums[left] > nums[pivot] and nums[right] < num
s[pivot]:
                self.swap(nums, left, right)
            if nums[left] <= nums[pivot]:
                left += 1
            if nums[right] >= nums[pivot]:
                right -= 1
```

```
        self.swap(nums, pivot, right)

        self.quickSort(nums, startIdx, right - 1)
        self.quickSort(nums, right + 1, endIdx)

    def swap(self, nums, left, right):
        nums[left], nums[right] = nums[right], nums[left]
```

What Changed and Why it Fixes TLE

	Your Version	Fixed Version
Pivot	Always `startIdx`	Random index
Worst case input	Sorted arrays → $O(n^2)$	Extremely unlikely → $O(n \log n)$ expected
Smaller side first	✅ kept but removed (negligible gain)	Not needed with random pivot

Randomization ensures no specific input can consistently cause worst-case behavior — LeetCode's sorted test cases were specifically triggering your $O(n^2)$ path.